

Producer-Consumer

Code: #include <stdio.h>

#include <pthread.h>

#include <unistd.h>

#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];

int in = 0, out = 0;

sem_t empty; // counts empty slots

sem_t full; // counts filled slots

pthread_mutex_t mutex; // mutual exclusion

void* producer(void* arg)

{

int item, i = 0;

while (1)

{

item = i++; // produce item

sem_wait(&empty);

pthread_mutex_lock(&mutex);

buffer[in] = item;

printf("Produced: %d at buffer[%d]\n", item, in);

in = (in + 1) % BUFFER_SIZE;

pthread_mutex_unlock(&mutex);

```

        sem_post(&full);

        sleep(1);
    }
}

void* consumer(void* arg)
{
    int item;

    while (1)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        sleep(2);
    }
}

int main()
{
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);

```

```

sem_init(&full, 0, 0);

pthread_mutex_init(&mutex, NULL);


pthread_create(&prod, NULL, producer, NULL);

pthread_create(&cons, NULL, consumer, NULL);


pthread_join(prod, NULL);

pthread_join(cons, NULL);


sem_destroy(&empty);

sem_destroy(&full);

pthread_mutex_destroy(&mutex);


return 0;
}

```

Output:

```

C:\Users\Admin\Desktop
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Produced: 2 at buffer[2]
Consumed: 1 from buffer[1]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[0]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[1]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[2]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[3]
Consumed: 8 from buffer[3]
Produced: 13 at buffer[0]
Consumed: 9 from buffer[4]
Produced: 14 at buffer[4]
Consumed: 10 from buffer[0]
Produced: 15 at buffer[0]
Consumed: 11 from buffer[1]
Produced: 16 at buffer[1]
Consumed: 12 from buffer[2]
Produced: 17 at buffer[2]
Consumed: 13 from buffer[3]
Produced: 18 at buffer[3]
Consumed: 14 from buffer[4]
Produced: 19 at buffer[4]

```

Dining Philosopher

Code: #include <stdio.h>

#include <pthread.h>

#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];

pthread_t philosophers[N];

void* philosopher(void* num)

{

int id = *(int*)num;

int left = id;

int right = (id + 1) % N;

while (1)

{

printf("Philosopher %d is thinking.\n", id);

sleep(1);

if (id % 2 == 0)

{

pthread_mutex_lock(&forks[left]);

printf("Philosopher %d picked left fork %d\n", id, left);

pthread_mutex_lock(&forks[right]);

printf("Philosopher %d picked right fork %d\n", id, right);

}

else

{

```

    pthread_mutex_lock(&forks[right]);
    printf("Philosopher %d picked right fork %d\n", id, right);

    pthread_mutex_lock(&forks[left]);
    printf("Philosopher %d picked left fork %d\n", id, left);
}

printf("Philosopher %d is eating.\n", id);
sleep(2);

pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);

printf("Philosopher %d put down forks %d and %d\n", id, left, right);
}

return NULL;
}

int main()
{
    int i;
    int ids[N];

    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    for (i = 0; i < N; i++)

```

```

{

pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);

}

for (i = 0; i < N; i++)

{

pthread_join(philosophers[i], NULL);

}

for (i = 0; i < N; i++)

{

pthread_mutex_destroy(&forks[i]);

}

return 0;

}

```

Output:

```

C:\Users\Admin\Desktop
Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 4 is thinking.
Philosopher 1 is thinking.
Philosopher 3 is thinking.
Philosopher 3 picked right fork 4
Philosopher 1 picked right fork 2
Philosopher 1 picked left fork 1
Philosopher 1 is eating.
Philosopher 3 picked left fork 3
Philosopher 3 is eating.
Philosopher 0 picked left fork 0
Philosopher 3 put down forks 2 and 4
Philosopher 4 picked left fork 4
Philosopher 2 picked left fork 2
Philosopher 2 picked right fork 3
Philosopher 2 is eating.
Philosopher 3 is thinking.
Philosopher 0 picked right fork 1
Philosopher 0 is eating.
Philosopher 1 put down forks 1 and 2
Philosopher 1 is thinking.
Philosopher 0 put down forks 0 and 1
Philosopher 0 is thinking.
Philosopher 2 put down forks 2 and 3
Philosopher 2 is thinking.
Philosopher 4 picked right fork 0
Philosopher 4 is eating.
Philosopher 1 picked right fork 2
Philosopher 1 picked left fork 1
Philosopher 1 is eating.
Philosopher 1 put down forks 1 and 2
Philosopher 1 is thinking.
Philosopher 3 picked right fork 4
Philosopher 3 picked left fork 3
Philosopher 3 is eating.
Philosopher 0 picked left fork 0
Philosopher 0 picked right fork 1
Philosopher 0 is eating.
Philosopher 2 picked left fork 2
Philosopher 4 put down forks 4 and 0
Philosopher 4 is thinking.
|

```